

伴游（TravelMate）出行旅游平台软件开发计划书

小组名称：Miracle

项目名称：伴游（TravelMate）出行旅游平台

版本号：V1.0

编写日期：2026年4月16日

1 引言

1.1 标识

本文档名称为《伴游（TravelMate）出行旅游平台软件开发计划书》，适用于“伴游（TravelMate）出行旅游平台”课程大作业项目。本文档用于说明本项目在计划阶段的软件开发安排、交付内容、开发方法、进度组织与资源配置。本文档版本号为 V1.0。

1.2 系统概述

“伴游（TravelMate）”是一款面向出行旅游场景的一体化平台，围绕“行前规划—行中服务—行后分享”的完整旅程展开，主要实现机票预订、火车票购买、酒店住宿、本地玩乐、AI 行程规划、AI 客服、旅途社区分享、后台管理与系统可观测等功能。系统对标携程、同程、去哪儿等真实旅游平台，强调核心需求覆盖、功能可用性、小并发稳定性以及过程文档与最终交付之间的一致性。

本项目为软件工程基础课程大作业，开发对象为课程模拟项目，面向教师与助教这一模拟“甲方”完成过程文档、系统实现、部署说明及答辩展示。

1.3 文档概述

本文档用于说明项目在课程周期内的软件开发总体安排，重点描述交付产品、所需工作、开发方法、产品标准、进度组织、团队分工与风险控制。本文档主要服务于项目组成员之间的协同开发，也作为教师和助教了解项目规划与监督项目实施过程的重要依据。

1.4 与其他计划之间的关系

本文档是项目过程文档体系中的计划阶段文档，与后续《软件需求规格说明书》《软件概要设计说明书》《软件详细设计说明书》《测试报告》《部署文档》《用户手册》相互衔接。

其中，本文档回答“做什么、何时做、由谁做、按什么标准做”；《软件需求规格说明书》回答“系统应实现什么”；《软件概要设计说明书》回答“系统如何分层和划分模块”；《软件详细设计说明书》回答“接口、数据结构和关键逻辑如何设计”；《测试报告》与《部署文档》则回答“系统如何验证与如何运行”。

2 引用文件

- 《软件工程基础2026春大作业要求》；
- 《软件开发计划书》模板；
- 《软工小组大作业》项目构思文档；
- 项目后续阶段形成的《软件需求规格说明书》《软件概要设计说明书》《软件详细设计说明书》《测试报告》《部署文档》《用户手册》；
- 项目开发过程中使用的接口文档、数据库设计稿、原型设计稿及 AI 使用说明 Markdown 附件。

3 交付产品

3.1 程序

本项目最终交付的程序产品包括以下内容：

- (1) 用户端前端程序，用于实现普通用户的机票、火车票、酒店、本地玩乐、AI 行程规划、AI 客服、社区浏览与内容发布等功能；
- (2) 管理端前端程序，用于实现资源管理、订单查看、内容审核、用户与权限管理、可观测仪表盘展示等功能；
- (3) 后端服务程序，用于提供认证授权、业务逻辑处理、订单管理、库存控制、消息通知、内容审核、AI 接口封装等 RESTful API；
- (4) 数据库初始化脚本，用于创建数据表、索引、初始测试数据与模拟业务数据；
- (5) 部署运行脚本与配置说明，用于指导教师或助教在验收环境中完成项目启动。

3.2 文档

本项目最终交付的文档包括：

- (1) 软件开发计划书；
- (2) 软件需求规格说明书；
- (3) 软件概要设计说明书；
- (4) 软件详细设计说明书；
- (5) 测试报告；
- (6) 部署文档；
- (7) 用户手册；
- (8) AI 使用说明 Markdown 附件，包括大模型在业务功能与辅助编码中的使用方式、Prompt、Session、Skills 等说明；
- (9) 演示视频与答辩展示材料。

3.3 非移交产品

以下成果属于项目开发过程中的中间产物或内部协作材料，不作为最终移交产品：

- (1) 草稿版原型图、头脑风暴记录、会议纪要；
- (2) 本地调试日志、临时测试脚本、临时 Mock 接口；
- (3) 开发分支上的中间版本代码；
- (4) 团队内部沟通记录、任务拆分表、非最终版数据库草图；
- (5) 各成员本地环境中的缓存文件、编译产物和依赖目录。

3.4 验收标准

本项目的验收以课程要求和最终答辩要求为依据，主要标准如下：

- (1) 核心需求覆盖完整，系统应覆盖出行旅游平台的主要核心需求，并在答辩时清晰说明各核心需求与模块对应关系；
- (2) 团队分工满足课程要求，平均每人至少负责 1 个核心需求、4 个模块，每个模块至少 5 个主要功能点；
- (3) 系统可用性良好，系统在教师或助教验收环境下可以正常启动、正常访问，功能效果与演示视频基本一致；
- (4) 系统具备小并发稳定性，对订票、订房等关键场景需具备基本防超卖和异常处理能力，不出现明显接口报错；
- (5) 过程文档连续且一致，计划书、需求、设计、测试、部署等文档之间应前后对应，与最终系统实现保持一致；
- (6) 若使用大模型，应提交单独 Markdown 文件说明其用途、Session、Prompt、Skills 以及人工校验情况。

3.5 最后交付期限

根据课程安排，本项目的主要交付期限如下：

- (1) 第 7 周：提交《软件开发计划书》；
- (2) 第 8 周：提交《软件需求规格说明书》；
- (3) 第 10 周：提交《软件概要设计说明书》；
- (4) 第 12 周：提交《软件详细设计说明书》；
- (5) 第 15 周：提交《测试报告》《部署文档》《用户手册》、源代码、可执行代码及演示材料。

4 所需工作概述

本项目围绕出行旅游平台这一课程选题展开，目标是在有限课程周期内完成一个可演示、可部署、可答辩的软件系统。项目的工作重点不是构建完整商业级 OTA 平台，而是在课程要求范围内完成核心需求识别、模块划分、系统设计、编码实现、联调测试和最终交付。

从需求角度看，系统需覆盖“行前规划—行中服务—行后分享”三类典型使用场景，并形成较完整的业务闭环。行前侧重点在于机票、火车票、酒店、本地玩乐与 AI 行程规划；行中侧重点在于订单履约、价格提醒、消息通知和 AI 客服；行后侧重点在于游记社区、互动社交与用户中心。同时，后台管理端还需承担资源维护、内容审核、角色权限管理和可观测仪表盘等管理职责。

从约束角度看，本项目必须满足课程对过程文档连续性、核心需求覆盖度、系统可用性、团队分工合理性和小并发稳定性的要求。因此，项目不追求真实商业支付、真实票务接口、真实地图服务或真实大规模数据接入，而采用“业务闭环真实、外部依赖可模拟、关键逻辑可验证”的实现策略。支付流程以模拟支付为主，票务与酒店资源以 Mock 数据和数据库数据为主，重点验证下单、库存扣减、订单流转、社区发布与 AI 功能集成等关键链路。

从文档角度看，项目需形成由计划书、需求规格说明书、概要设计说明书、详细设计说明书、测试报告、部署文档和用户手册组成的文档体系。各文档之间应保持一致：计划书中的阶段安排应在后续文档中得到落实，需求文档中的功能应在设计文档中完成映射，设计文档中的接口和数据结构应在代码与测试中得到体现。

从生命周期角度看，本项目处于课程驱动的软件开发生命周期中，当前阶段属于从方案规划到编码交付的完整开发周期。项目主要经历“需求调研与竞品分析—计划阶段—需求分析—概要设计—详细设计—编码实现—联调测试—部署验收”这一过程。

从资源与进度角度看，项目采取五人协同开发方式。团队成员分别负责大交通票务系统、酒店与目的地服务系统、AI 行程规划与 Agent 助手、旅途社区与用户中心、超级管理后台与可观测系统五个核心方向。项目采用分工明确、接口先行、阶段评审、集中联调的组织方式推进，减少后期接口不一致和数据库设计割裂带来的返工风险。

从开发策略角度看，项目优先采用成熟技术栈与可重用基础组件，在不照搬开源旅游项目的前提下复用通用框架、UI 组件库、鉴权组件、可视化组件与缓存中间件，避免在登录注册、基础管理和通用 CRUD 等低价值环节上投入过多时间。项目强调“自研核心业务逻辑，合理复用通用基础设施”，以满足课程对原创性、可解释性与工程完整性的要求。

从质量与风险控制角度看，项目将重点关注以下问题：一是需求过宽导致实现不完整；二是数据库主键、外键和订单模型不统一导致联调失败；三是订票订房场景中的库存超卖；四是 AI 模块存在接口不稳定、输出不可控或无法解释的问题；五是社区与后台的权限控制不到位。为此，项目将在计划阶段固化分工，在设计阶段优先统一数据模型与接口规范，在开发阶段通过代码评审、日志记录、异常处理和小并发测试控制风险。

5 实施整个软件开发活动的计划

5.1 软件开发过程

本项目采用“里程碑驱动的迭代式开发过程”。在总体上遵循需求分析、系统设计、详细设计、编码实现、测试部署的顺序推进；在具体执行上允许各成员围绕自己负责模块进行并行开发，并在统一的数据结构和接口规范下逐步联调。

项目的主要开发阶段安排如下：

第一阶段为前期调研与原型准备。团队通过调研携程、同程、去哪儿及内容社区类产品，抽取典型业务链路，明确系统定位、核心需求和界面风格，并建立基础数据集、原型草图与代码仓库。

第二阶段为计划阶段。团队整理项目目标、交付范围、分工、阶段成果和风险控制方式，形成《软件开发计划书》，作为后续工作的总控文档。

第三阶段为需求分析阶段。团队从普通用户和管理员两个角色出发，识别核心用例，形成功能需求、非功能需求和边界约束，提交《软件需求规格说明书》。此阶段重点回答“系统必须做什么”。

第四阶段为概要设计阶段。团队完成系统分层架构、模块划分、数据库 E-R 图、核心表结构和模块间交互关系设计，提交《软件概要设计说明书》。此阶段重点回答“系统整体如何组织”。

第五阶段为详细设计阶段。团队完成关键接口定义、时序图、类或数据结构设计、订单流转规则、库存控制方案、AI 调用流程及消息机制设计，提交《软件详细设计说明书》。此阶段重点回答“系统关键功能如何落地”。

第六阶段为编码与联调阶段。各成员根据既定分工完成前后端编码，逐步完成自测、模块联调、数据库联调和全系统联调，解决库存扣减、权限鉴权、消息提醒、内容审核与 AI 调用等关键问题。

第七阶段为测试、部署与验收准备阶段。团队围绕功能正确性、接口可用性、关键流程稳定性和部署可复现性开展测试，整理测试报告、部署文档、用户手册、演示视频与答辩 PPT，并完成课程验收。

5.2 软件开发总体状态

5.2.1 软件开发方法

本项目采用“需求驱动、模块划分、接口先行、迭代集成”的开发方法。

在需求阶段，团队使用用例分析和场景分析方法梳理普通用户与管理的关键行为，围绕核心业务链路识别功能边界。对于出行旅游平台而言，重点场景包括航班与车次查询、酒店预订、订单生成、价格提醒、AI 行程规划、AI 客服问答、社区内容发布与后台审核等。

在设计阶段，项目采用分层架构与模块化设计方法。整体上划分为表现层、业务层、数据层与外部服务接入层；业务上划分为票务、酒店、AI、社区、后台五个核心方向。设计时优先统一用户、订单、资源、消息、内容等核心实体及其主外键关系，避免后续模块间割裂。

在实现阶段，项目采用前后端分离开发方法。前端负责页面展现、交互逻辑与状态管理，后端负责业务编排、权限校验、数据持久化和接口输出。接口层采用 RESTful 风格，接口定义优先通过接口文档工具进行约定，再进入编码实现。对尚未完成的后端接口可先使用 Mock 数据支撑前端开发。

在协作阶段，项目采用 Git 分支协同开发方法。主分支用于稳定版本提交，各成员在个人功能分支上开发，经自测与合并检查后再并入主干。数据库结构调整、公共枚举变化、公共接口修改等事项必须通过小组讨论后统一执行。

在测试阶段，项目采用“模块自测 + 关键流程联调 + 小并发验证”的方法。普通功能以黑盒测试为主，关键业务如订票、订房、订单支付和库存扣减增加异常输入测试与并发场景验证，保证系统在课程验收环境下保持稳定。

在 AI 功能实现方面，项目采用“模型调用封装 + 技能调用 + 结果人工校验”的方法。AI 行程规划与 AI 客服由独立服务层统一封装，调用支持函数调用或工具调用能力的大模型接口；涉及航班、天气、酒店、行程等信息时，通过 Skills 调用内部业务接口或模拟工具接口完成问答增强。所有与大模型相关的用途、Prompt、Session 和 Skills 均记录在单独 Markdown 附件中，确保可解释、可追溯。

本项目主要使用的手工与自动化工具包括：需求整理工具、原型设计工具、接口文档工具、数据库建模工具、Git 代码托管工具、IDE 开发工具、JMeter 压力测试工具、日志监控与可视化组件等。

5.2.2 软件产品标准

本项目在需求表达、设计描述、编码实现、测试用例和测试结果记录方面统一采用规范化标准，以确保不同成员提交的代码和文档风格一致、便于联调和答辩。

（一）需求与设计文档标准

需求文档采用“功能需求 + 非功能需求 + 角色/用例 + 约束条件”的写法，避免将实现细节提前写入需求。概要设计文档重点说明系统架构、模块划分、数据库设计和关键业务流程；详细设计文档重点说明接口定义、时序图、关键算法、状态流转和异常处理。所有文档中的术语、模块命名、表名、接口名应与代码实现尽量保持一致。

（二）前端编码标准

前端主要采用 Vue 3 + Vite 技术栈。组件文件采用 PascalCase 命名，页面路由与目录名称采用小写短横线风格，变量与函数采用 camelCase 命名。每个页面组件应按“模板、脚本、样式”分区组织，复杂页面应拆分为多个可复用子组件。样式命名尽量语义化，禁止大面积写无意义缩写。对涉及接口调用、表单校验、状态切换、列表筛选等关键逻辑，应在必要位置补充注释。

（三）后端编码标准

后端主要采用 Java 17 + Spring Boot 3.x + MyBatis-Plus 技术栈。包命名采用分层方式组织，如 controller、service、service.impl、mapper、entity、dto、vo、config、util 等。类名采用 PascalCase，方法名和变量名采用 camelCase，常量名采用全大写下划线风格。控制器仅负责参数接收与结果返回，不直接编写复杂业务；业务逻辑统一收敛于 service 层；数据库访问统一通过 mapper 层完成。统一定义返回结果格式、异常处理机制和状态码规范。

（四）数据库与 SQL 标准

数据表命名采用小写下划线风格，主键统一为 id 或业务明确的唯一标识，时间字段建议统一为 create_time、update_time。外键字段命名应能清晰体现关联对象，如 user_id、order_id、hotel_id、post_id。SQL 设计应避免无索引的高频查询，关键查询条件应建立必要索引。库存扣减、订单状态更新等语句应考虑并发安全性与幂等性。

（五）注释与文档化标准

每个核心类、核心接口和复杂方法应说明其用途、输入输出、主要约束和异常情况。对于 AI 调用封装、推荐排序规则、库存扣减策略、敏感词审核逻辑等不易从代码表面直接理解的功能，必须提供较完整注释或补充到设计文档中。

（六）测试标准

测试用例采用“用例编号—前置条件—操作步骤—预期结果—实际结果”的格式进行记录。测试内容覆盖正常流程、异常流程、边界输入和关键并发场景。测试结果应可追踪到具体功能模块，问题修复后应进行回归测试。

（七）复杂度与实现约束

为保证课程项目的可维护性，项目禁止无规划地堆叠复杂框架，禁止直接照搬现成旅游平台开源项目。单个方法不宜过长，复杂业务应进行适当拆分。公共能力优先抽象为公共组件或公共服务，避免在多个模块中复制粘贴相同逻辑。

5.2.3 可重用的软件产品

本项目允许并鼓励在合理范围内吸纳成熟、通用、可解释的可重用软件产品，以提高开发效率和代码质量，但严禁直接照搬完整旅游平台业务项目。

本项目拟吸纳的可重用软件产品主要包括：前端 UI 组件库、图表可视化库、后端开发框架、ORM 框架、缓存中间件客户端、JWT 鉴权组件、文件上传组件、日志组件、测试工具及支持大模型接入的官方 SDK。对这些产品的评估标准主要包括：一是是否成熟稳定；二是是否与既定技术栈兼容；三是是否有清晰文档；四是是否便于二次开发；五是不会导致课程项目原创性不足。

在使用可重用产品时，团队将坚持以下原则：基础框架和通用组件可以复用，核心业务逻辑必须自主实现。诸如下单流程、库存扣减、订单状态机、社区互动规则、内容审核流程、AI 行程规划与客服技能编排等内容，均由团队自行分析设计与编码实现。

在项目实现过程中，团队也会主动沉淀可在多个模块中复用的通用能力，以减少重复开发和后期维护成本。计划沉淀的可重用能力包括：统一登录鉴权组件、统一返回结果封装、分页查询组件、上传组件、消息通知组件、订单状态枚举与转换工具、通用审核流程、日志记录工具、AI 服务调用封装、前端通用卡片与列表组件等。

5.2.4 处理关键性需求

本项目重点处理的关键性需求包括安全性、稳定性、一致性与可解释性。

在安全性方面，系统将通过参数校验、统一异常处理、身份鉴权、角色权限控制、关键操作日志记录、文件上传格式校验、敏感词过滤和输入合法性检查等方式降低常见风险。对订票、订房等关键场景，系统将采取库存判断、并发控制和订单状态校验等方式避免明显的逻辑错误和超卖问题。

在稳定性方面，系统将重点保证抢票、抢房、订单支付、内容发布和后台审核等核心流程在小并发场景下能够正常运行。团队将通过乐观锁、库存校验、异常回滚、日志定位和回归测试保证关键业务稳定。

在一致性方面，项目将确保计划书、需求文档、设计文档、数据库设计、接口定义、代码实现和答辩展示之间前后对应，避免“文档一套、系统一套”。

在可解释性方面，特别是 AI 模块和可观测模块，必须能够向教师和助教清楚说明设计思路、技能调用机制和数据来源。

5.2.5 计算机硬件资源利用

本项目的开发主要依赖团队成员个人计算机完成，硬件资源需求总体可控。前端开发、后端开发、数据库运行和本地联调均可在普通开发电脑上完成，无需专用服务器或高性能 GPU 集群。

项目运行阶段的主要软件与硬件资源包括：开发电脑、代码托管平台、Java 运行环境、Node.js 环境、MySQL 数据库、Redis 缓存、接口调试工具、浏览器、测试工具与必要的模型调用网络环境。在资源利用上，团队将优先采用本地运行与轻量级部署方案，尽量降低环境搭建成本。对于库存并发验证和接口压力验证，仅进行课程规模的小并发测试，不开展大规模生产级压测。

5.2.6 记录原则

本项目中的“关键决策”是指对系统方向、数据模型、公共接口、技术路线和高风险实现策略产生显著影响的决策。例如：是否采用前后端分离架构、订单表与资源表如何关联、库存扣减采用何种策略、AI 功能采用何种模型接入方式、后台审核与权限如何实现等。

这些关键决策将记录在以下位置：项目计划书、需求与设计文档、数据库设计稿、接口文档、代码仓库提交记录和小组会议纪要。记录原则是“关键事项可追溯、重要变更有依据、最终版本能解释”，以便在答辩、联调和问题修复时快速回溯设计原因。

7 进度表和活动安排

7.1 进度表

阶段	周次	主要任务	主要成果物	负责人
前期准备	第1-6周	竞品调研、需求梳理、原型设计、技术选型、仓库初始化	调研记录、原型草图、技术路线初稿	全体
计划阶段	第7周	明确交付范围、团队分工、风险与进度安排	软件开发计划书	邹林利、全体
需求阶段	第8周	梳理角色、用例、功能需求和非功能需求	软件需求规格说明书	全体
概要设计阶段	第9-10周	完成系统架构设计、模块划分、数据库 E-R 图和核心表结构	软件概要设计说明书	陈一鸿、李科、全体
详细设计阶段	第11-12周	完成接口定义、时序图、关键业务流程设计	软件详细设计说明书	全体
开发与联调阶段	第13-14周	完成前后端编码、自测、联调、关键并发场景验证	可运行系统、联调记录	全体
测试与验收阶段	第15周	完成功能测试、部署验证、答辩材料整理	测试报告、部署文档、用户手册、演示视频、源代码	全体

7.2 活动关系说明

本项目活动之间具有明显的先后依赖关系。计划阶段为需求与设计阶段提供总体约束；需求阶段决定概要设计和详细设计的边界；概要设计为详细设计、数据库建模和接口拆分提供基础；详细设计完成后进入大规模编码与联调；测试和部署工作建立在系统核心功能基本完成的前提之上。

与此同时，前端页面开发、后端接口开发和数据库准备可以在统一规范下并行推进。为了降低后期返工风险，项目将在概要设计阶段优先确定用户表、订单表、资源表、社区内容表和消息表等关键数据结构，并在详细设计阶段统一接口契约。

8 项目组织和资源

8.1 项目组织

本项目采用五人协同开发模式，根据课程要求保证平均每人至少负责 1 个核心需求、至少 4 个模块、每个模块至少 5 个主要功能点。具体分工如下：

A 邹林利：大交通票务与订单系统

模块1：机票预订（查询、航司筛选、余票实时显示、退改签政策查看、一键下单）

模块2：火车票预订（车次实时查询、席别选择、中转方案推荐、常用乘车人管理、余票监控）

模块3：价格比对与订阅（多平台模拟比价、历史价格走势图中、低价提醒订阅、特价日历、优惠券应用）

模块4：基础订单管理（订单状态流转控制、模拟支付接口对接、电子票据生成、行程单导出、退款处理逻辑）

B 莫谨瑞：目的地住宿与本地生活系统

模块1：酒店搜索与展示（多维度搜索、商圈/地标筛选、酒店详情与设施清单、地图找房模式、实时房价看板）
模块2：房务预订逻辑（入住日期动态选择、多房型管理、住客信息自动填充、房态防超卖控制、延时退房申请）
模块3：景点与本地玩乐（景点门票预订、一日游产品展示、周边服务推荐、电子核销二维码生成、景区热度提示）
模块4：评价与反馈体系（五星评分系统、图文评价上传、标签化评价分类、评价点赞与举报、商家回复管理）

C 陈一鸿：AI 智能规划与 Agent 服务

模块1：AI 智能行程规划（个性化需求输入、调用大语言模型生成行程、每日路线可视化、行程预算估算、行程一键转购物车）
模块2：AI 贴心客服 Agent（自然语言意图识别、智能问答库检索、Skills/工具调用、多轮对话状态保持、人工客服转接）
模块3：实时通知服务（航班/车次变更提醒、酒店入住倒计时、系统站内信通知、红点未读逻辑、通知分类管理）
模块4：动态行程管理器（多同行共享行程、行程节点拖拽修改、关联订单快速跳转、足迹地图生成、行程导出 PDF）

D 杜新城：旅途社区与用户中心

模块1：UGC 内容发布（多图上传与裁剪、地理位置打卡、话题标签关联、动态权限设置、存为草稿功能）
模块2：社区瀑布流交互（双列瀑布流展示、内容搜索与过滤、关注/推荐流切换、视频/图文兼容显示、热门榜单计算）
模块3：社交关系链（关注/粉丝体系、点赞与收藏夹管理、二级评论树状结构、私信即时通讯、用户拉黑功能）
模块4：用户中心与鉴权（JWT 安全登录注册、个人资料精美编辑、账号安全绑定、行为日志记录、用户等级与勋章系统）

E 李科：管理后台与可观测性系统

模块1：数据可观测仪表盘（实时订单大屏、热门目的地统计、系统访问量 QPS 监控、报错日志实时看板、财务结算图表）
模块2：资源库管理（航班/火车基础数据维护、酒店房态强制干预、景点信息审核与上架、促销活动配置中心、库存预警管理）
模块3：内容与安全审核（社区图文敏感词自动过滤、人工审核 workflow、违规动态一键屏蔽、举报任务处理、黑名单管理）
模块4：系统架构控制（RBAC 角色权限控制、系统操作日志审计、接口限流与熔断配置、数据库备份管理、多端 API 密钥管理）

在协作关系上，A 与 B 重点围绕交易履约和库存模型协同；C 与 A/B 之间需要共享订单、行程和消息相关接口；D 与 E 之间需要对接内容审核、用户权限和举报处理流程；全体成员在用户体系、订单中心、消息中心和数据库设计上保持统一。

8.2 项目资源

本项目所需资源主要包括以下几类：

- （1）人力资源：项目由 5 名成员共同完成，采用按核心方向分工、按阶段集中联调的协同方式推进；
- （2）开发环境资源：团队成员个人计算机、IDE、浏览器、Git 仓库、JDK 17、Node.js、MySQL 8.0、Redis；
- （3）设计与协作资源：原型设计工具、接口文档工具、数据库建模工具、即时沟通工具；
- （4）测试资源：Postman/Apifox、JMeter、本地日志与调试工具；
- （5）AI 资源：大模型 API Key、Prompt 调试记录、Skills 设计与调用封装；
- （6）交付资源：数据库初始化脚本、部署脚本、演示视频、答辩 PPT。

9 风险管理

本项目的主要风险及应对措施如下：

9.1 需求范围过大

出行旅游平台天然功能较多，若前期需求扩张过快，容易导致后期无法按时完成。对此，团队将优先保证核心需求落地，将扩展功能控制在课程时间允许范围内，确保先完成主链路，再完善亮点功能。

9.2 数据模型不统一

如果用户、订单、资源、消息、社区内容等关键实体设计不统一，后期联调会出现大量返工。对此，团队将在概要设计阶段统一数据库 E-R 图、主外键命名规则和关键表结构，并冻结核心字段。

9.3 库存超卖与并发异常

订票订房场景存在明显的并发风险。对此，团队将在后端采用库存校验、乐观锁或原子更新语句等方式控制并发，并通过小并发测试验证关键流程稳定性。

9.4 AI 模块输出不稳定

AI 行程规划与智能客服可能出现结果不准确、格式不稳定或接口调用失败等问题。对此，团队将对模型调用结果增加格式约束、兜底提示和人工校验，并将 Prompt、Skills 与使用留痕写入单独 Markdown 文档，保证可解释性。

9.5 前后端联调延期

若前后端接口定义过晚或频繁变动，将直接影响开发进度。对此，团队将采用接口先行策略，在详细设计阶段统一关键接口，未完成部分使用 Mock 数据支撑页面开发。

9.6 验收环境无法复现

若系统部署步骤复杂、依赖缺失或配置不明确，助教验收时容易失败。对此，团队将在交付前整理数据库初始化脚本、运行说明和部署文档，尽量做到环境清晰、步骤明确、可重复启动。

10 注解

10.1 缩略语说明

- OTA: Online Travel Agency, 在线旅游代理平台
- UGC: User Generated Content, 用户生成内容
- JWT: JSON Web Token, 一种常见的鉴权方式
- RBAC: Role-Based Access Control, 基于角色的访问控制
- QPS: Queries Per Second, 每秒请求数
- Mock: 模拟数据或模拟接口
- API: Application Programming Interface, 应用程序接口
- Skills: 大模型调用的外部工具或函数能力

10.2 说明

本计划书仅覆盖本课程大作业从需求分析、系统设计、编码实现、测试验收到部署交付的开发过程。更细化的软件需求分析、系统设计、详细接口、测试用例和部署步骤将在后续阶段文档中展开。